

AGENTOPS: ENABLING OBSERVABILITY OF LLM AGENTS

Liming Dong, Qinghua Lu, Liming Zhu
Data61, CSIRO, Australia

December 3, 2024

ABSTRACT

Large language model (LLM) agents have demonstrated remarkable capabilities across various domains, gaining extensive attention from academia and industry. However, these agents raise significant concerns on AI safety due to their autonomous and non-deterministic behavior, as well as continuous evolving nature. From a DevOps perspective, enabling observability in agents is necessary to ensuring AI safety, as stakeholders can gain insights into the agents’ inner workings, allowing them to proactively understand the agents, detect anomalies, and prevent potential failures. Therefore, in this paper, we present a comprehensive taxonomy of AgentOps, identifying the artifacts and associated data that should be traced throughout the entire lifecycle of agents to achieve effective observability. The taxonomy is developed based on a systematic mapping study of existing AgentOps tools. Our taxonomy serves as a reference template for developers to design and implement AgentOps infrastructure that supports monitoring, logging, and analytics. thereby ensuring AI safety.

1 Introduction

An large language model (LLM) is a large-scale language model with tens of billions of parameters, pretrained on vast and diverse datasets, and applicable to various downstream tasks [1]. While LLMs demonstrate impressive capabilities, they also exhibit limitations in understanding and performing complex tasks. This has led to an increasing demand for LLM agents (including agentic systems which are designed to prompt an LLM multiple times using agent-like design patterns and exhibit varying degrees of agent-like behavior), which have the ability to handle complex tasks autonomously [2].

An LLM agent is an autonomous system powered by LLMs, capable of perceiving context, reasoning, planning, executing workflows, by leveraging external tools, knowledge bases and other agents to achieve human goals [3]. LLM agents have shown remarkable potential to enhance productivity across various domains, attracting widespread attention from academia and industry. For example, many agents are being successfully applied in the software engineering domain, such as Devin ¹, ChatDev ², SWE-agent ³. Hereafter, we use ”agents” to refer specifically to LLM agents throughout this paper.

Despite the huge potential to enhance productivity, the adoption of LLM agents introduces unique challenges due to their inherent characteristics.

- **Complex Artefacts and Pipelines:** The agents are compound AI systems, integrating LLMs with various components (i.e., design time artifacts), such as context engine and external tools, and dynamically generating runtime artifacts, such as goals and plans. The operational pipelines typically include context processing, reasoning and planning, workflow execution, and continuous evolution based on the feedback. Throughout these processes, the pipelines may leverage external tools, knowledge bases, and other agents to achieve human goals.

¹Devin, <https://www.cognition.ai/blog/introducing-devin>

²ChatDev. <https://github.com/OpenBMB/ChatDev>

³SWE-agent, <https://github.com/princeton-nlp/SWE-agent>

AgentOps: 实现大语言模型智能体的可观测性

董立明、卢清华、朱立明 Data61, 澳大利亚联邦科学与工业研究组织 (CSIRO), 澳大利亚

2024年12月3日

摘要

大语言模型（LLM）智能体已在多个领域展现出卓越的能力，并受到学术界和工业界的广泛关注。然而，由于其自主性和非确定性行为，以及持续演化的特性，这些智能体引发了关于AI安全的重大担忧。从DevOps的角度来看，为智能体实现可观测性对于确保AI安全至关重要，因为利益相关者可以深入了解智能体的内部运作，从而主动理解智能体、检测异常并防止潜在故障。因此，本文提出了一个全面的AgentOps分类法，识别出为了在整个智能体生命周期中实现有效可观测性，需要追踪的工件及相关数据。该分类法基于对现有AgentOps工具的系统映射研究而开发。我们的分类法为开发者设计和支持监控、日志记录和分析的AgentOps基础设施提供了参考模板，从而确保AI安全。

1 引言

大型语言模型（LLM）是指参数量达数百亿、在海量且多样的数据集上预训练、并适用于各种下游任务的语言模型 [1]。尽管大语言模型表现出令人印象深刻的能力，但它们在理解和执行复杂任务方面仍存在局限性。这导致了对能够自主处理复杂任务的大语言模型智能体（包括智能体系统，其采用类似智能体的设计模式多次提示大语言模型，并表现出不同程度的智能体行为）日益增长的需求 [2]。

大语言模型智能体是一种由大语言模型驱动的自主系统，能够通过利用外部工具、知识库及其他智能体来感知上下文、进行推理、规划并执行工作流，从而实现人类目标 [3]。大语言模型智能体在提升各领域生产力方面展现出显著潜力，引发了学术界和工业界的广泛关注。例如，许多智能体已成功应用于软件工程领域，如 Devin ¹、ChatDev ²，和 SWE-agent ³。在本文的其余部分，我们使用“智能体”一词特指大语言模型智能体。

尽管大语言模型智能体在提升生产力方面具有巨大潜力，但其固有特性也带来了独特的挑战。

- **复杂工件与流水线：**智能体是复合AI系统，将大语言模型与各种组件（即设计时工件）（如上下文引擎和外部工具）相结合，并动态生成运行时工件（如目标和计划）。运营流水线通常包括基于反馈的上下文处理、推理与规划、工作流执行以及持续进化。在这些过程中，流水线可能会利用外部工具、知识库和其他智能体来实现人类目标。

¹Devin, <https://www.cognition.ai/blog/introducing-devin>

ChatDev. <https://github.com/OpenBMB/ChatDev>

³SWE-agent, <https://github.com/princeton-nlp/SWE-agent>

- **Autonomy:** The agents operate with a high degree of autonomy, dynamically interact with external environments, including shifting context, external knowledge bases and tools. These interactions are not predetermined, which may increase the risk of unintended behaviors (such as selecting an external tool with vulnerability issues) and pose severe AI safety challenges [4].
- **Non-Deterministic Behaviour:** Due to the probabilistic nature of LLMs, the agents often exhibit non-deterministic behaviour, producing varied outputs even when given the same inputs. This lack of repeatability may lead to unintended consequences, making it challenging to ensure consistent and predictable outcomes.
- **Continuous Evolution:** The agents can evolve over time through continuous learning, adapting based on runtime evaluation results or human feedback. While this adaptability enhance the agents’ capabilities and skills, it also introduces further challenges in maintaining alignment with intended quality and safety goals over the agent’s lifecycle.
- **Shared Accountability:** The responsibility of the behaviour or decisions of the agents is often shared among multiple stakeholders, including the agent owner, the FM provider, and various providers of external tools/agents. This complicates the identification of failure sources and the assignment of accountability in the event of incidents.

To address the challenges outlined above, it is essential for DevOps tools to support observability features, enabling stakeholders to monitor agent behaviour, track the status of artifacts, log associated data, detect anomalies, trace the evolution of artifacts, and assign accountability if incidents occur. **Observability** refers to the ability to gain actionable insights into the inner workings of an agent by analysing the inputs and outputs (i.e., runtime artifacts) of different components (i.e., design time artifacts), as they flow through the operational pipelines [5]. However, most existing DevOps tools for agents primarily focus on LLM-specific metrics and prompt management, with limited support for the observability of agent-specific artifacts such as goals, plans, and tools. This limitation results in insufficient observability from both the system and pipeline perspectives.

To bridge this gap, we propose **AgentOps**, a specialised DevOps paradigm tailored for agents. AgentOps provides a holistic view of agent operations, enabling comprehensive observability by systematically tracing of agent artifacts and associated data. In this paper, we perform a systematic mapping study on the existing DevOps tools for monitoring agents and/or agentic systems to understand their features and limitations. Based on the study results, we first propose an artifact relationship model to identify key agent artifacts and their relationship. Then we present a comprehensive taxonomy of AgentOps, detailing the artifacts and associated data that should be traced. Our taxonomy serves as a reference template for developers to design and implement AgentOps tools to support monitoring, logging, and analytics of agents.

The remainder of the paper is organized as follows. Section 2 introduces the methodology. Section 3 presents the mapping study results of AgentOps tools and core features. Section 4 identifies the agent artifacts and their relationship and presents the taxonomy of AgentOps. Section 5 addresses threats to validity, and Section 6 concludes the paper and outlines future work.

2 Methodology

In this section, we introduce the methodology for this study. Figure1 provides the overview of the search process. Following the guidelines outlined in [6, 7], we performed a systematic mapping study to analyse the existing tools related to AgentOps. This study aimed to understand their features and limitations and to identify the agent artifacts and associated data that should be traced to enable observability.

2.1 Data Sources

To identify tools relevant to AgentOps, we used multiple sources. Initially, we conducted a comprehensive search on GitHub⁴ using carefully selected keywords. The repositories were then filtered based on predefined selection criteria to ensure relevance and quality. To complement the GitHub search and address potential gaps, we performed a targeted Google Search⁵ to capture additional tools that may not have been available or visible in the GitHub search results. This multi-source approach ensured broad coverage of both open-source and proprietary AgentOps relevant tools.

- 自主性：智能体以高度的自主性运行，与外部动态环境进行交互，包括不断变化的上下文、外部知识库和工具。这些交互并非预先确定，这可能会增加意外行为（例如选择存在漏洞问题的外部工具）的风险，并带来严重的AI安全挑战 [4]。
- 非确定性行为：由于大语言模型的概率特性，智能体通常表现出非确定性行为，即使在给定相同输入时也会产生不同的输出。这种缺乏可重复性可能导致意外后果，使得确保一致且可预测的结果变得具有挑战性。
- 持续进化：智能体能够通过持续学习随时间演进，基于运行时评估结果或人类反馈进行自适应调整。尽管这种适应性增强了智能体的能力与技能，但也为其生命周期内维持与预期质量和安全目标的一致性带来了进一步挑战。
- 共同责任：智能体的行为或决策的责任通常由多个利益相关者共同承担，包括智能体所有者、基础模型提供商以及各类外部工具/智能体的提供商。这增加了在事故发生时确定故障源和分配问责制的复杂性。

为应对上述挑战，DevOps工具必须支持可观测性功能，使利益相关者能够监控智能体的行为、追踪工件的状态、记录相关数据、检测异常、追踪工件的演变，并在事故发生时进行问责。可观测性是指通过分析不同组件（即设计时工件）的输入和输出（即运行时工件）在运营流水线中的流动，来获得关于智能体内部运作的可操作的见解 [5]。然而，大多数现有的用于智能体的DevOps工具主要关注大语言模型特定指标和提示管理，对智能体特定工件（如目标、计划和工具）的可观测性支持有限。这种局限性导致从系统与流水线视角来看，可观测性不足。

为弥合这一差距，我们提出了AgentOps，一种专为智能体定制的DevOps范式。AgentOps提供了对智能体运营的整体视图，通过对智能体工件及其相关数据的系统性追踪，实现了全面的可观测性。在本文中，我们对现有用于监控智能体和/或智能体系统的DevOps工具进行了系统映射研究，以了解其功能特性和局限性。基于映射研究结果，我们首先提出了一个工件关系模型，以识别关键智能体工件及其关系。然后，我们提出了AgentOps的全面分类法，详细说明了应被追踪的工件及相关数据。我们的分类法可作为开发者设计和实现AgentOps工具的参考模板，以支持对智能体的监控、日志记录和分析。

本文其余部分组织如下。第2节介绍方法论。第3节展示AgentOps工具及核心功能的映射研究结果。第4节识别智能体工件及其关系，并提出AgentOps的分类法。第5节讨论效度威胁，第6节总结全文并概述未来工作。

2 方法论

在本节中，我们介绍了本研究的{方法论}。图1提供了搜索过程的概览。按照 [6, 7], 中概述的指南，我们执行了一次系统映射研究，以分析与AgentOps相关的现有工具。这项研究的目的是了解这些工具的功能特性和局限性，并识别应进行追踪的智能体工件及相关数据，以实现可观测性。

2.1 数据源

为了识别与AgentOps相关的工具，我们使用了多种来源。最初，我们在GitHub⁴ 上进行了全面搜索，使用了精心挑选的关键词。随后，我们根据预定义的选择标准对仓库进行筛选，以确保相关性和质量。为了补充GitHub搜索并弥补潜在缺口，我们执行了有针对性的谷歌搜索⁵，以捕捉那些可能在GitHub搜索结果中不可用或不可见的额外工具。这种多源方法确保了对开源和专有AgentOps相关工具的广泛覆盖。

⁴Github.<https://github.com>

⁵Google Search.<https://google.com>

⁴Github.<https://github.com> ⁵谷歌搜

索.<https://google.com>

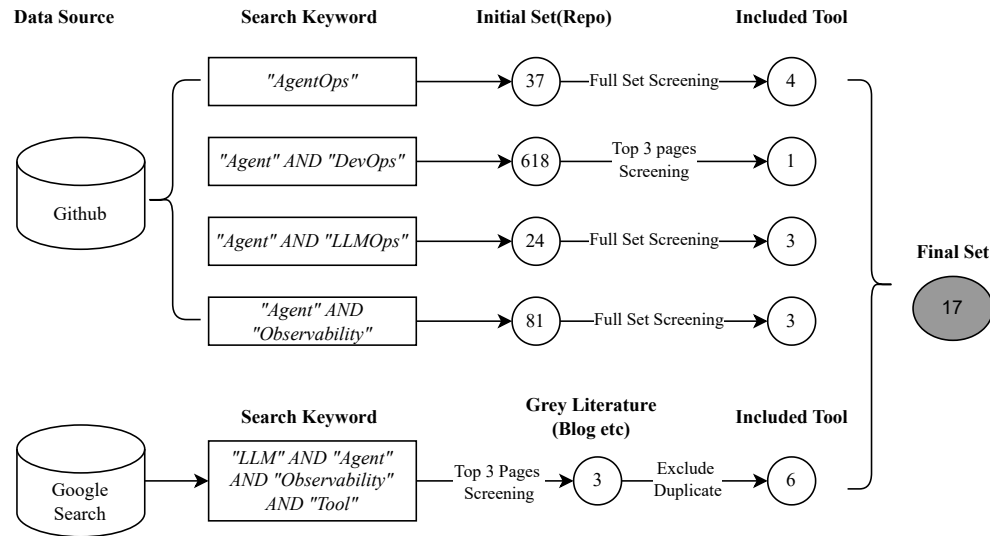


Figure 1: Search Process of AgentOps Relevant Tools

2.2 Search String

The search terms we used in GitHub primarily focused on the keywords ((*AgentOps*) OR (*Agent* AND *DevOps*) OR (*Agent* AND *LLMops*)). Since the concept of AgentOps and the exploration of related tools are still in the early stages, we extended the scope of our search to include observability tools that provide comprehensive visibility into LLM applications, must including agents tracing and observability features as well. As a result, we also incorporated the search term (*Agent* AND *Observability*). The search keyword term used in Google Search is (*LLM* AND *Agent* AND *Observability* AND *Tool*).

Table 1: Selection Criteria

ID	Inclusion Criteria
I1	Support observability features (such as monitoring and tracing).
I2	Support agent specific tracing or LLM application tracing that can be applied to agents, not just LLM-level tracing.
I3	Formal release version.
I4	Public online document available.

2.3 Selection Criteria

We conducted the tool selection process from the multi-channel data sources mentioned above. For this paper, we focused on AgentOps relevant tools that met the criteria illustrated in Table 1. To ensure robust tracking and management of agent performance and interactions, it is essential that selected tools support core observability functionalities (*I1*) like monitoring and tracing etc. Furthermore, given that the focus of our study is on AgentOps rather than general LLMops or model-level operations, the tools must support tracing at the agent level or LLM application level (*I2*) that allows us to observe a finer-grained understanding of agent-specific behaviors. To facilitate both implementation and community-wide adoption, it is essential that tools with formal release version (*I3*) and have accessible and public online documentation (*I4*).

2.4 Search Process

We began by querying GitHub with four specific keyword strings *AgentOps*, *Agent* AND *DevOps*, *Agent* AND *LLMops*, and *Agent* AND *Observability*. Each query returned an initial set of repositories. For the search strings (*AgentOps*) OR (*Agent* AND *LLMops*) OR (*Agent* AND *Observability*), we obtained 37, 24, and 81 initial repositories, respectively. All repositories from these search results were screened to ensure relevance, leading to the inclusion of 4, 3, and 3 tools for further analysis, respectively.

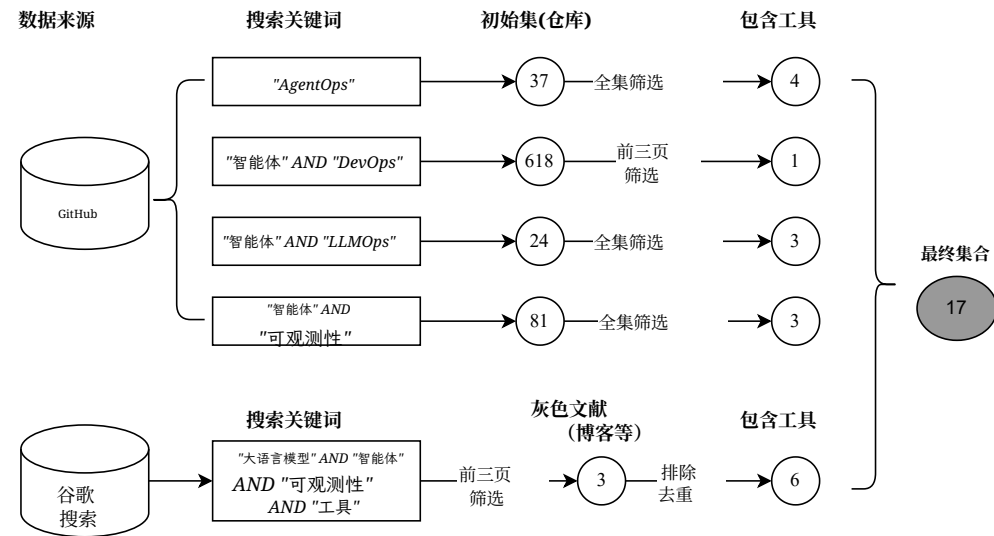


图 1: AgentOps 相关工具的搜索过程

2.2 搜索字符串

我们在 GitHub 上使用的搜索条件主要侧重于以下关键词：((*AgentOps*) OR (*Agent* AND *DevOps*) OR (*Agent* AND *LLMops*)). 由于 AgentOps 概念及相关工具的探索仍处于早期阶段，我们扩大了搜索范围，纳入能够为 LLM 应用提供全面可视性的可观测性工具，其中必须包含智能体追踪和可观测性功能。因此，我们还引入了搜索词 (*Agent* AND *Observability*)。在谷歌搜索中使用的搜索关键词是 (*LLM* AND *Agent* AND *Observability* AND *Tool*)。

表 1: 选择标准

ID	纳入标准
I1	支持可观测性功能（如监控和追踪）。
I2	支持智能体特定追踪或可应用于的大语言模型应用追踪智能体，而不仅仅是大语言模型层面的追踪。
I3	正式发布版本。
I4	公开在线文档可用。

2.3 选择标准

我们从上述多渠道数据源中执行了工具选择过程。在本研究中，我们重点关注符合表1所示标准的与 AgentOps 相关的工具。为了确保对智能体性能和交互的稳健跟踪与管理，所选工具必须支持核心可观测性功能（如监控和追踪）（*I1*）。此外，鉴于我们的研究重点是 AgentOps，而非一般的 LLMops 或模型级操作，工具必须支持智能体级别或 LLM 应用级别的追踪（*I2*），以便我们能够更细致地观察和理解智能体特定行为。为了促进实施和社区的广泛采用，工具必须具备正式发布版本（*I3*），并提供可访问的公开在线文档（*I4*）。

2.4 搜索过程

我们首先使用四个特定的关键词字符串对 GitHub 进行查询：*AgentOps*、*Agent* 和 *DevOps*、*Agent* 和 *LLMops*，以及 *Agent* 和 *Observability*。每个查询都返回了一个初始集的仓库。对于搜索字符串 (*AgentOps*) 或 (*Agent* 和 *LLMops*) 或 (*Agent* 和 *Observability*)，我们分别获得了 37、24 和 81 个初始仓库。对所有来自这些搜索结果中的仓库进行了筛选，以确保相关性，最终分别纳入了 4、3 和 3 个工具以进行进一步的分析。

For the (“*Agent*” AND “*DevOps*”) query, the search yielded a much larger set of 618 repositories. To manage this, we limited our screening to the top three pages of results, sorting repositories based on relevance and popularity. Most of these projects, however, were focused on agents for DevOps tasks rather than DevOps platforms for managing agents. As a result, only one additional tool was included from this query.

To ensure comprehensive coverage and address potential gaps in the GitHub search, we conducted an additional search using Google Search to capture proprietary tools. We used the keyword string (“*LLM*” AND “*Agent*” AND “*Observability*” AND “*Tool*”) and screened the top three pages of results. This search targeted relevant blogs, product announcements, and other online resources. After removing duplicates and irrelevant results, this process identified six additional tools.

The combined search process across GitHub and Google Search resulted in a total of 17 tools. This final set formed the basis for our systematic analysis of AgentOps relevant tools’ features and limitations.

2.5 Data Extraction

To better understand the current landscape of AgentOps tools, we systematically categorized and summarized 17 selected tools. Relevant data items were extracted from each tool’s GitHub repository, official product website, and available project documentation. The specific data items extracted are outlined in Table 2.

Table 2: Data Extraction

ID	Data Item	Description
D1	Name	The name of AgentOps relevant tool.
D2	Source	The data source used to identify the included tool.
D3	GitHub Repo URL	The Github repository URL of included tool.
D4	Star	The number of stars received on Github (as a proxy for popularity).
D5	Scope	The scope of the tool, specifically whether it is designed for tracing agents or LLM applications.
D6	Key Features	The primary features provided by the tool for AgentOps or LLM application observability.
D7	Traceable Artifacts	The artifacts and associated data that the tool tracks.

3 Mapping Study Result

In this section, we present the results of the systematic mapping study on tools relevant to AgentOps.

Table 3: List of Tools relevant to AgentOps

Name	Source	Github Repo URL	Star	Scope
Agenta	Github	Agenta-AI/agenta	1.3k	LLM applications
AgentNeo	Github	raga-ai-hubAgentNeo	1k	Agents
AgentOps	Github	AgentOps-AI/agentops	2.1k	Agents
AGIFlow	Github	AgiFlow/agiflow-sdks	21	Agents
Arize	Google	Arize-Phoenix	3.9k	LLM applications
DataDog	Github	DataDog/datadog-agent	2.9k	Agents
Dify	Github	langgenius/dify	51.6k	LLM applications
Helicone	Github	Helicone/helicone	1.9k	LLM applications
Laminar	Github	lmnr-ai/lmnr	1.1k	LLM applications
Langfuse	Github	langfuse/langfuse	6.5k	LLM applications
LangSmith	Github	langchain-ai/langsmith-sdk	417	LLM applications
LangTrace	Github	Scale3-Labs/langtrace	552	LLM applications
Lunary	Google	lunary-ai/lunary	1.1k	LLM applications
PortKey	Google	Portkey-AI/gateway	6.3k	LLM applications
TraceLoop	Google	traceloop/openllmetry	3.4k	LLM applications
Trulens	Google	truera/trulens	2.2k	LLM applications

对于“（'Agent' 且 'DevOps'）”的查询，搜索返回了数量大得多的618个仓库。为了管理这一情况，我们将筛选范围限制在结果的前三页，并根据相关性和流行度对仓库进行排序。然而，这些项目中的大多数专注于面向 DevOps任务的智能体，而非用于管理智能体的DevOps平台。因此，从该查询中仅额外纳入了一项工具。

为确保全面覆盖并弥补GitHub搜索中可能存在的空白，我们使用谷歌搜索进行了一次额外的搜索，以捕获专有工具。我们使用了关键词字符串（“LLM”和“Agent”和“Observability”和“Tool”），并筛选了搜索结果的前三页。该搜索针对相关博客、产品公告及其他在线资源。在去重并排除无关结果后，此过程识别出六个额外工具。

通过 GitHub 和谷歌搜索相结合的方式进行检索，共识别出 17 个工具。该最终工具集构成了我们对 AgentOps 相关工具的功能特性与局限性进行系统分析的基础。

2.5 数据提取

为了更好地理解当前 AgentOps 工具的现状，我们对选定的 17 种工具进行了系统性的分类和总结。我们从每种工具的 GitHub 仓库、官方产品网站以及可用的项目文档中提取了相关数据项。所提取的特定数据项如表 2 所示。

表 2：数据提取

ID	数据项	描述
D1	Name	AgentOps 相关工具的名称。
D2	来源	用于识别包含工具的数据来源。
D3	GitHub 仓库 URL	包含工具的 GitHub 仓库 URL。
D4	Star	在 GitHub 上获得的星标数（作为流行度的代理指标）。
D5	范围	工具的范围，特别是其设计用途是用于跟踪智能体还是 LLM 应用。
D6	关键功能	该工具为 AgentOps 或 LLM 应用可观测性提供的主要功能特性。
D7	可追踪工件	工具所追踪的工件及关联数据。

3 映射研究结果

在本文的 本节，我们展示了与 AgentOps 相关的工具的系统映射研究的结果。

表 3：与 AgentOps 相关的工具列表

Name	来源	GitHub 仓库 URL	Star	范围
Agenta	GitHub	Agenta-AI/agenta	1.3k	LLM 应用
AgentNeo	GitHub	raga-ai-hubAgentNeo	1k	智能体
AgentOps	GitHub	AgentOps-AI/agentops	2.1k	智能体
AGIFlow	Github	AgiFlow/agiflow-sdks	21	智能体
Arize	谷歌	Arize-Phoenix	3.9k	LLM 应用
DataDog	Github	DataDog/datadog-agent	2.9k	智能体
Dify	GitHub	langgenius/dify	51.6k	LLM 应用
Helicone	Github	Helicone/helicone	1.9k	LLM 应用
Laminar	Github	lmnr-ai/lmnr	1.1k	LLM 应用
Langfuse	GitHub	langfuse/langfuse	6.5k	LLM 应用
LangSmith	GitHub	langchain-ai/langsmith-sdk	417	LLM 应用
LangTrace	GitHub	Scale3-Labs/langtrace	552	LLM 应用
Lunary	谷歌	lunary-ai/lunary	1.1k	LLM 应用
PortKey	谷歌	Portkey-AI/gateway	6.3k	LLM 应用
TraceLoop	谷歌	traceloop/openllmetry	3.4k	LLM 应用
Trulens	谷歌	truera/trulens	2.2k	LLM 应用

3.1 AgentOps-Relevant Tools

This study includes a diverse range of 17 tools relevant to AgentOps, designed to enable observability in agents or LLM applications. Table 3 highlights the AgentOps relevant tools currently available in the market. An analysis of GitHub star rankings reveal that many of these tools have received significant attention, with 14 out of 17 tools accumulating thousands of stars. As of November 2024, several tools stand out as leaders in their respective categories, such as AgentOps (1.7k stars) and AgentNeo (1k stars). Among tools designed for LLM applications, Langfuse (6.5k stars), PortKey (6.3k stars), and Arize (3.9k stars), TraceLoop (3.4k stars), and DataDog (2.9k stars) stand out as leading solutions. It is worth noting that some tools may have been unintentionally excluded due to keyword mismatches or incomplete documentation. We aim to address this limitation by monitoring emerging tools and incorporating them into future work.

Table 4: Key Features of AgentOps relevant Tools

Tool	Customisation	Prompt Mgmt	Evaluation	Feedback	Monitoring	Tracing	Guardrails
Agenta	Yes	Yes	Yes	Yes	Yes	Yes	No
AgentNeo	Yes	No	Yes	No	Yes	Yes	No
AgentOps	Yes	Yes	Yes	Yes	Yes	Yes	Yes
AGIFlow	Yes	Yes	Yes	No	Yes	Yes	No
Arize	No	No	Yes	No	Yes	Yes	Yes
Datadog	No	No	No	No	Yes	Yes	No
Dify	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Helicone	No	No	Yes	No	Yes	Yes	No
Langfuse	No	Yes	Yes	Yes	Yes	Yes	No
LangTrace	No	No	Yes	No	Yes	Yes	No
LangSmith	No	Yes	Yes	Yes	Yes	Yes	Yes
Lunary	No	Yes	Yes	Yes	Yes	Yes	Yes
TraceLoop	No	No	No	No	Yes	Yes	No
Trulens	No	Yes	Yes	Yes	Yes	Yes	Yes
Portkey	No	No	Yes	No	Yes	Yes	No

3.2 Key Features

We summarized the key features of the identified AgentOps relevant tool, as shown in Table 4 and Table 5.

3.2.1 Customization

When creating agents, existing tools extend agent capabilities by adding toolkits from marketplaces to agent workflows, connecting to multiple knowledge bases to improve performance, and integrating customized fine-tuned models for specific business use cases.

3.2.2 Prompt Management

Prompt versioning and management allow developers to store and track different versions of prompts, making it invaluable for testing, optimizing, and reusing prompts across various stages of agent production. The prompt playground enables developers to edit, import, and test various prompt templates with different models, helping to compare performance before deployment. Additionally, consistent monitoring of prompts is also necessary for maintaining the reliability and security of agents, particularly in detecting and mitigating issues such as code injection attacks and secret leaks embedded within prompts.

3.2.3 Evaluation

Evaluation is the process of assessing the behaviour and capabilities of an agent against specific criteria or general benchmarks. The typical evaluation process involves creating a suitable evaluation dataset, defining clear criteria and metrics, and conducting thorough testing based on these predefined metrics. It is important to test the agent’s performance against user requirements, standard leaderboards or comparable systems. For agents, evaluation goes beyond simply assessing the the final output. It is equally important to monitor and track the agent’s execution steps and evaluate intermediate outputs to ensure the entire process meets the intended goals and aligns with governance requirements.

3.1 AgentOps相关工具

本项研究涵盖了 17 种与 AgentOps 相关的多样化工具，旨在为智能体或 LLM 应用提供可观测性。表 3 突出展示了当前市场上可用的与 AgentOps 相关的工具。对 GitHub 星标排名的分析显示，其中许多工具已获得大量关注，17 个工具中有 14 个积累了数千个星标。截至 2024 年，若干工具在其各自类别中脱颖而出，成为领导者，例如 AgentOps（1.7k 星标数）和 AgentNeo（1k 星标数）。在专为 LLM 应用设计的工具中，Langfuse（6.5k 星标数）、PortKey（6.3k 星标数）和 Arize（3.9k 星标数）、TraceLoop（3.4k 星标数）以及 DataDog（2.9k 星标数）作为领先方案脱颖而出。值得注意的是，部分工具可能因关键词不匹配或文档不完整而被无意中排除。我们旨在通过监控新兴工具并将其纳入未来工作，来解决这一局限性。

表 4: AgentOps相关工具的关键功能

Tool	自定义	提示词管理	评估	反馈	监控	追踪	护栏
Agenta	Yes	Yes	Yes	Yes	Yes	Yes	No
AgentNeo	Yes	No	Yes	No	Yes	Yes	No
AgentOps	Yes	Yes	Yes	Yes	Yes	Yes	Yes
AGIFlow	Yes	Yes	Yes	No	Yes	Yes	No
Arize	No	No	Yes	No	Yes	Yes	Yes
Datadog	No	No	No	No	Yes	Yes	No
Dify	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Helicone	No	No	Yes	No	Yes	Yes	No
Langfuse	No	Yes	Yes	Yes	Yes	Yes	No
LangTrace	No	No	Yes	No	Yes	Yes	No
LangSmith	No	Yes	Yes	Yes	Yes	Yes	Yes
Lunary	No	Yes	Yes	Yes	Yes	Yes	Yes
TraceLoop	No	No	No	No	Yes	Yes	No
Trulens	No	Yes	Yes	Yes	Yes	Yes	Yes
PortKey	No	No	Yes	No	Yes	Yes	No

3.2 关键功能

我们总 概括了已识别的与AgentOps相关的工具的关键功能，如表4和表 5.

3.2.1 定制

在创建智能体时，现有工具通过从市场添加工具包到智能体 workflow、连接多个知识库以提升性能，以及集成针对特定业务场景定制的微调模型，来扩展智能体的能力。

3.2.2 提示管理

提示词版本控制和管理使开发者能够存储和跟踪不同版本的提示词，在智能体生产环境的各个阶段测试、优化和复用提示词时具有极高价值。提示词实验场允许开发者编辑、导入并使用不同模型测试各种提示词模板，从而在部署前比较性能。此外，对提示词进行持续监控对于维护智能体的可靠性和安全性也至关重要，尤其是在检测并缓解嵌入在提示词中的代码注入攻击和密钥泄露等问题时。

3.2.3 评估

评估是指根据特定标准或通用基准来衡量智能体的行为与能力。典型的评估流程包括创建合适的评估数据集、定义清晰的评估标准和指标，以及基于这些预先定义的指标进行详尽的测试。将智能体的性能与用户需求、标准排行榜或可比系统进行比较测试至关重要。对于智能体而言，评估不仅仅停留在最终输出的层面。同样重要的是监控并追踪智能体的执行步骤，评估中间输出，以确保整个流程符合预期目标并满足治理要求。

Table 5: AgentOps Relevant Tools Key Features

Category	Features	Description
Customization	Provision, custom, spawn, and deploy autonomous agents	Create customisable and scalable autonomous agents.
	Extend agent capabilities with toolkits	Add toolkits from marketplace to agent workflows.
	Extend agent capabilities with multiple vector databases	Connect to multiple vector databases to improve agent’s performance.
	Extend agent capabilities with fine-tuned models	Custom fine-tuned models for business specific use cases.
Prompt Management	Prompt versioning and management	Keep track of different versions of prompts used in agents. Useful for A/B testing and optimizing agent performance.
	Prompt playground with model comparisons	Test and compare different prompts and models for agents before deployment.
	Prompt injection detection	Identify potential code injection and secret leaks.
Evaluation	Test agents against benchmarks and leaderboards.	Create a dataset, define metrics, run evaluations, compare results, track results over time etc.
	Evaluate agents in diverse step	Evaluate Final Response- Evaluate the agent’s final response.
		Evaluate Single step-Evaluate any agent step in isolation (<i>e.g.</i> , whether it selects the appropriate tool).
		Evaluate Trajectory- Evaluate whether the agent took the expected path (<i>e.g.</i> , of tool calls) to arrive at the final answer.
Feedback	Collect explicit feedback	Directly prompt the user to give feedback, this can be a thumb up or a thumb down.
	Collect implicit feedback	Measure the user’s behavior, this can be time spent on a page, click-through rate.
Monitoring	Agent analytics dashboard	Monitor diverse level and dimension statistics metrics about agents.
	LLM cost management and tracking	Track spend (token cost) with foundation model providers.
Tracing	LLM/agent tracing	Trace each agent span, <i>e.g.</i> , the whole chain, retrieval, LLM call, tool call etc.
		Trace evaluation span
		Trace user feedback
Guardrails	Predefined rules and constraints	Set rules or constraints to limit agent actions, ensuring safe and predictable behavior.
	Fallback and escalation paths	Provide safe defaults or redirect cases to human operators in ambiguous or risky scenarios.

LangSmith⁶ introduces two additional dimensions of agent evaluation: 1) Step-by-Step Evaluation: Assess each individual step the agent takes in isolation, such as determining whether it selects the appropriate tool. 2) Trajectory Evaluation: Examine whether the agent followed the expected sequence of actions, including the series of tool calls, to arrive at the final answer. This ensures that the decision-making process is sound, not just the outcome.

3.2.4 Feedback

Human feedback plays a key role in evaluating the quality of an agent’s output. Feedback is collected as a score and attached to an execution trace or an individual LLM generation. Feedback can be also used to retrain/fine-tune LLM or improve the design of agents (e.g. stored in memory or used in prompts as positive or negative examples). Langfuse⁷ defined different types of feedback that can be collected that vary in quality, detail, and quantity: 1) Explicit Feedback: Directly prompt the user to give feedback, this can be a rating, a like, a dislike, a scale or a comment. While it is simple to implement, quality and quantity of the feedback is often low. More structured and fine-grained feedback is expected. 2) Implicit Feedback: Measure the user’s behavior, *e.g.*, time spent on a page, click-through rate, accepting or rejecting final output. This type of feedback is more difficult to implement but is often more frequent and reliable.

3.2.5 Monitoring

Developers can continuously monitor agent performance and enhance observability throughout the agent’s execution process by closely tracking its outputs. This involves keeping track of monitoring metrics (*e.g.*, latency and cost),

表5: AgentOps 相关工具关键功能

类别	功能特性	描述
定制	配置、自定义、生成以及部署自主智能体	创建可定制且可扩展的自主智能体。
	扩展智能体功能工具包	扩展智能体能力。将工具包从市场添加到智能体 workflow。
	使用以下功能扩展智能体能力多个向量数据库	连接多个向量数据库以提升智能体的性能。
	使用以下功能扩展智能体能力微调模型	用于特定业务用例的自定义微调模型。
提示管理	提示词版本控制与管理	跟踪智能体中使用的不同版本的提示词。有助于进行A/B测试并优化智能体性能。
	带有模型对比的提示词实验场对比	在部署前测试并比较智能体的不同提示词和模型。
	提示注入检测	识别潜在的代码注入和密钥泄露。
评估	根据基准测试和排行榜测试智能体。	创建数据集，定义度量指标，运行评估，比较结果，跟踪随时间变化的结果等。
	在多种步骤中评估智能体	评估最终响应- 评估智能体的最终响应。
		评估单步- 单独评估智能体的任何一步（例如，是否选择了适当的工具）。
		评估轨迹- 评估智能体是否按照预期的路径（例如，工具调用路径）得出最终答案。
反馈	收集显式反馈	直接提示用户给出反馈，可以是点赞或点踩。
	收集隐式反馈	测量用户行为，这可以是页面停留时间、点击率。
监控	智能体分析仪表盘	监控智能体不同层级和维度的统计指标。
	大语言模型成本管理与跟踪	跟踪与基础模型提供商的支出（令牌成本）。
追踪	LLM/智能体追踪	追踪每个智能体跨度，例如整个链路、检索、LLM 调用、工具调用等。
		追踪评估跨度
		追踪用户反馈
护栏	预定义规则和约束	设置规则或约束条件以限制智能体动作，确保安全且可预测的行为。
	回退和升级路径	提供安全默认值，或在模糊或存在风险的场景中 将案例重定向给人类操作员。

LangSmith⁶ 引入了智能体评估的两个额外维度：1) 逐步评估：独立评估智能体所采取的每一个具体步骤，例如判断其是否选择了合适的工具。2) 轨迹评估：考察智能体是否按照预期的行动序列（包括一系列工具调用）得出了最终答案。这确保了决策过程的严谨性，而不仅仅是结果的正确性。

3.2.4 反馈

人类反馈在评估智能体输出质量方面起着关键作用。反馈以评分的形式收集，并附加到执行轨迹或单次大语言模型生成上。反馈还可以用于重新训练/微调大语言模型或改进智能体的设计（例如存储在记忆或作为正面或反面示例用于提示词）。Langfuse⁷定义了可收集的不同类型的反馈，这些反馈在质量、细节和数量上各不相同：1) 显式反馈：直接提示用户给出反馈，可以是评分、点赞、点踩、量表或评论。虽然实现简单，但反馈的质量和数量往往较低。通常期望获得更具结构化和细粒度的反馈。2) 隐式反馈：衡量用户行为，例如页面停留时间、点击率、接受或拒绝最终输出。这种类型的反馈较难实现，但通常更频繁且可靠。

3.2.5 监控

通过密切跟踪智能体的输出，开发者可以持续监控智能体性能并在整个执行过程中增强可观测性。这包括跟踪监控指标（例如，延迟和成本），

⁶LangSmith.<https://docs.smith.langchain.com/evaluation> ⁷ Langfuse.<https://langfuse.com/docs/scores/user-feedback>

associating feedback with agent spans to evaluate performance, and debugging issues by diving into specific traces and spans where errors occurred. Monitoring also helps identify the root causes of unexpected results, errors, or latency issues, allowing developers to optimize performance based on real-time feedback.

3.2.6 Tracing

AgentOps is designed to support developers in transitioning from prototype to production, ensuring that the work does not stop once the agent is created and initial tests are passed. Within the otool, agents execute increasingly complex tasks and iterative runs, such as chains, tool-assisted agents, and advanced prompts. By adding traces, AgentOps captures the entire process—from the moment a user sends a prompt to the final output—helping developers understand each step and identify the root causes of any issues. Execution tracing allows developers to follow the agent’s decision-making process step-by-step, providing insights into the flow of actions, decisions, and interactions. This helps identify where errors or unexpected behaviors originate, making it easier to isolate and address problems within complex agentic workflows. There is no doubt that tracing is the most direct way to enable observability in AgentOps platform. All tools listed in the Table 3 have implemented tracing functionality.

3.2.7 Guardrails

Guardrails for agent application are essential for ensuring AI-safety-by-design. Arize have already integrated Arize Guards⁸. Additionally, an increasing number of tools (*e.g.*, AgentNeo⁹) have Guardrails implementation on their planned feature lists, aiming to enhance the safety of agents. AgentOps tools can trace the activation, execution process and outcomes of guardrails, which can be used to generate safety cases for auditing purposes.

4 Taxonomy of AgentOps

Once agent developer have deployed their agent, they need trace its execution process to monitor agent performance and evolution of agent artifacts. The AgentOps tool provides tracing functions to troubleshoot performance and correlate data throughout the product, enabling developer to find and resolve issues in agents.

In this section, we first introduce an entity-relationship model to describe various relationships between different agent artifacts. Then, we present a comprehensive taxonomy of Agentops, which serves as a template for developers to design and implement AgentOps tools for tracing agent artifacts and their associate data.

4.1 Entity-Relationship Model for Agent Artifacts

To clarify the relationships between traceable artifacts, Figure 2 illustrates the connections among nested spans in an agent trace.

When an **agent** receives a user **goal**, it can utilize zero or more **knowledge bases** to gather information or support decision-making. These knowledge bases supply essential data and contextual information for the agent span. A single **reasoning** span generates a **plan**, structured through logical processes and analysis.

An **agent** can generate multiple plan spans, each representing a structured plan to address specific objectives. The **agent** relies on these plans to organize and coordinate its tasks. Each **plan** span may call one or more **LLM** spans to leverage the LLM’s processing capabilities, aiding in the execution of planned actions. A **plan** is realized as a single Workflow, translating the strategic plan into practical execution.

A **workflow** comprises multiple **tasks**, with each **task** representing a specific action within the larger **workflow**. These **task** collectively fulfill the goals outlined in the **workflow**. Each **task** may utilise one or more **tools**, providing additional functionality or resources essential for task execution. **Task** may also call one or more **LLM** spans to access model-based functionalities, such as prediction or analysis, that are critical for task completion.

Evaluation spans assess either a specific **agent**, a **plan** or a single **workflow**, ensuring that the agent’s overall performance or the effectiveness of the workflow aligns with intended goals. The **guardrail** monitors all other spans in the agent’s lifecycle, enforcing constraints and ensuring compliance with predefined rules. This universal connection helps maintain the safety of the agent’s actions across all spans.

将反馈与智能体跨度关联以评估性能，并深入调查发生错误的特定追踪和跨度以调试问题。监控还有助于识别意外结果、错误或延迟问题的根本原因，从而让开发者基于实时反馈优化性能。

3.2.6 追踪

AgentOps 旨在支持开发者从原型过渡到生产环境，确保在智能体创建并通过初始测试后，工作不会就此停止。在 otool 中，智能体执行日益复杂的任务和迭代运行，例如链条、工具辅助代理以及高级提示。通过添加追踪，AgentOps 捕获整个流程——从用户发送提示到最终输出——帮助开发者理解每一步骤，并识别任何问题的根本原因。执行追踪使开发者能够逐步跟踪智能体的决策过程，从而深入了解操作、决策和交互的流程。这有助于确定错误或意外行为的来源，使复杂代理工作流中的问题更容易被隔离和解决。毫无疑问，追踪是在 AgentOps 平台上实现可观测性的最直接方式。表 3 中列出的所有工具均已实现追踪功能。

3.2.7 护栏

智能体应用的护栏对于确保设计即AI安全至关重要。Arize 已集成 Arize Guards⁸。此外，越来越多的工具（例如 AgentNeo⁹）在其计划功能列表中包含了护栏实现，旨在增强智能体的安全性。AgentOps 工具可以追踪护栏的激活、执行过程和结果，从而用于生成用于审计的安全案例。

4 AgentOps分类法

一旦智能体开发者部署了他们的智能体，就需要追踪其执行过程，以监控智能体性能及智能体工件的演进。AgentOps 工具提供追踪功能，用于排查性能问题并关联产品全链路数据，帮助开发者发现并解决智能体中的问题。

在本节中，我们首先介绍一个实体关系模型，用于描述不同智能体工件之间的关系。随后，我们提出了一个全面的AgentOps分类法，作为开发者设计和实现AgentOps工具的模板，以追踪智能体工件及其关联数据。

4.1 用于智能体工件的实体关系模型

为阐明可追踪工件之间的关系，图 2 展示了智能体追踪中嵌套跨度之间的连接。

当智能体接收用户目标时，它可以利用零个或多个知识库来收集信息或支持决策。这些知识库为智能体的跨度提供关键数据和上下文信息。单个推理跨度会通过逻辑过程和分析生成一个计划。

智能体可以生成多个计划跨度，每个计划跨度代表一个针对特定目标的结构化计划。智能体依赖这些计划来组织和协调其任务。每个计划跨度可以调用一个或多个LLM跨度，以利用大语言模型的处理能力，辅助执行计划中的操作。计划被实现为单一的工作流，将战略计划转化为实际执行。

工作流由多个任务组成，每个任务代表较大工作流中的特定操作。这些任务共同实现工作流中概述的目标。每个任务可能使用一个或多个工具，为任务执行提供所需的额外功能或资源。任务还可以调用一个或多个大语言模型跨度，以访问对任务完成至关重要的基于模型的功能，例如预测或分析。

评估跨度用于评估特定智能体、计划或单个工作流，以确保智能体的整体性能或工作流的有效性符合预期目标。护栏监控智能体生命周期中的其他所有跨度，执行约束条件并确保符合预定义规则。这种通用连接有助于在所有跨度中保持智能体操作的安全性。

⁸Arize Guards. <https://docs.arize.com/arize/llm-monitoring-and-guardrails/guardrails>

⁹AgentNeo. <https://github.com/raga-ai-hub/agentneo>

⁸Arize 护栏。 <https://docs.arize.com/arize/llm-monitoring-and-guardrails/guardrails>

⁹AgentNeo。 <https://github.com/raga-ai-hub/agentneo>

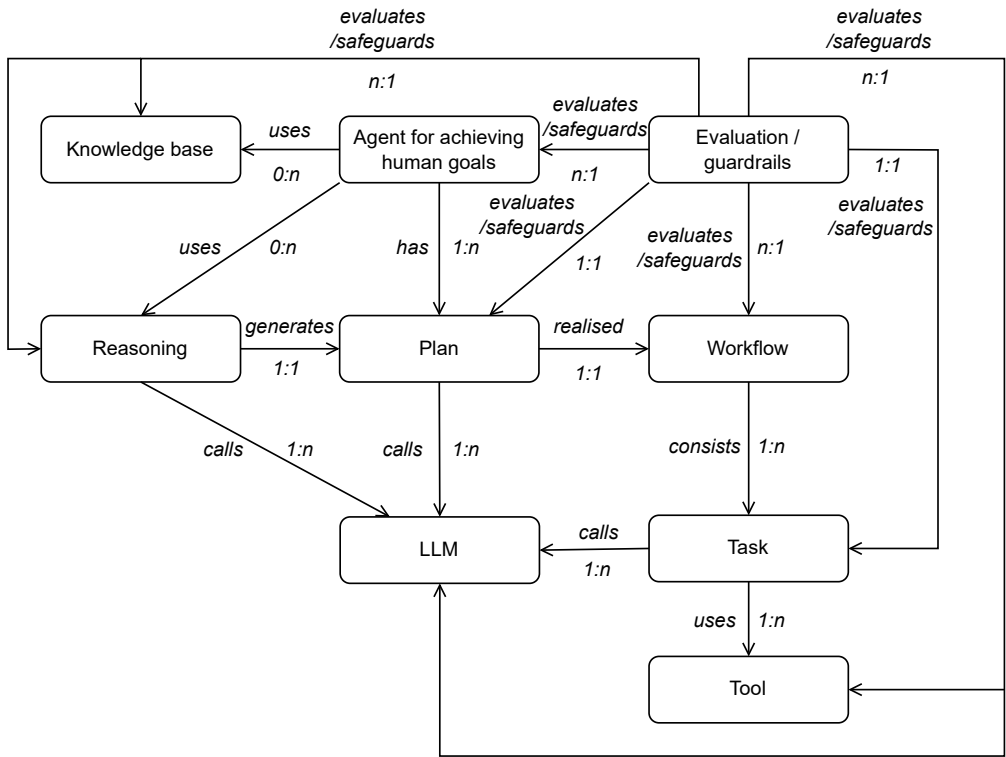


Figure 2: Entity-Relationship Model for Agent Artifacts

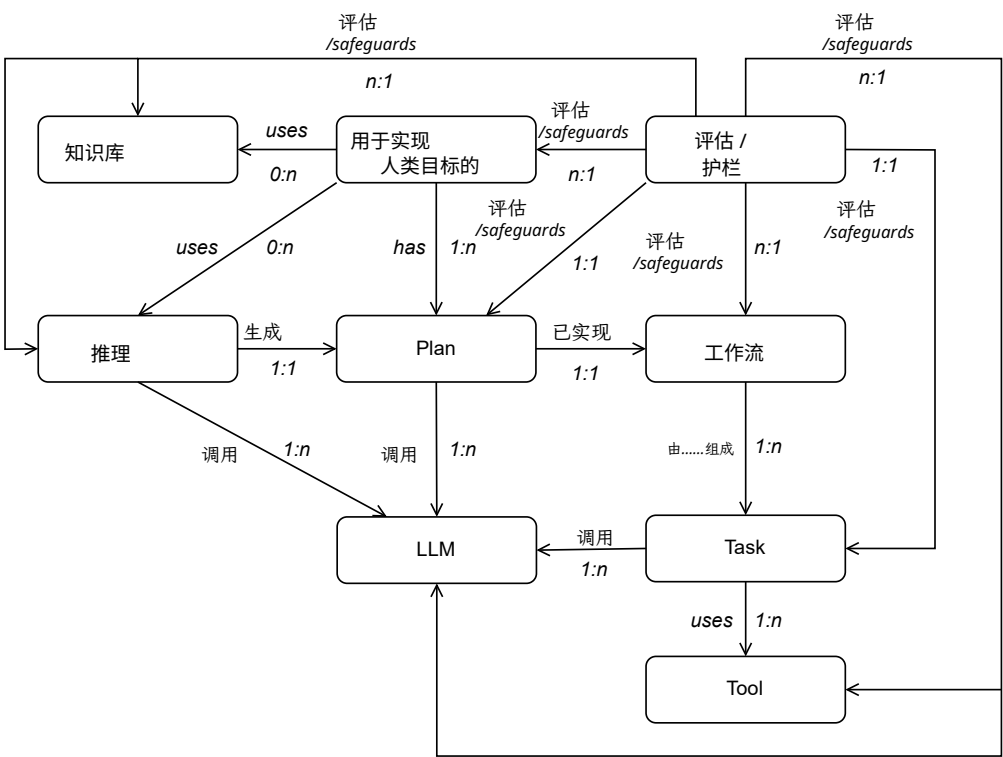


图 2：智能体工件的实体关系模型

4.2 Taxonomy of AgentOps

In AgentOps, a **trace** reveals the entire process from the moment a user submits a goal achieving request to the point when the final result is delivered. This includes the reasoning process, the plan generated, the workflow along with its associated tasks, the knowledge retrieved, the tools invoked, the evaluation and guardrails applied, and multiple LLM calls.

A **trace** consists of one or more **spans**. The first span represents the root span. Each root span represents a request from start to finish. The spans beneath the parent provide deeper context for what occurs during a request, detailing the steps that make up the request.

Aspan could consists of the following metadata attributes:

- **Name:** The label or identifier of the span, indicating the type of operation being performed.
- **Start Timestamp:** The exact time when the span begins, providing temporal context for tracking and performance analysis.
- **Duration:** The total time taken for the span’s operation, measured from start to finish. This metric helps in analyzing efficiency and identifying potential performance bottlenecks.
- **Attributes:**
 - **Inputs and Outputs:** Data fed into the span (e.g., user goal) and the resulting outputs (e.g., tool calling result or final result).
 - **Error Type, Message, and Traceback:** Information about any errors encountered, including the error type, a descriptive message, and traceback details for debugging.
 - **Metrics:** Quantitative data related to the span, such as input_tokens, output_tokens, evaluation metrics, monitoring metrics, which measure and track cost usage and agent performance.

4.2 AgentOps分类法

在AgentOps中，追踪揭示了从用户提交目标达成请求到最终结果交付的整个过程。这包括推理过程、生成的计划、工作流及其关联任务、检索的知识、调用的工具、应用的评估与护栏，以及多次大语言模型调用。

一条追踪由一个或多个跨度组成。第一个跨度表示根跨度。每个根跨度代表一个从开始到结束的请求。位于父级之下的跨度为请求期间发生的情况提供更深入的上下文，详细说明了构成请求的各个步骤。

跨度可以包含以下元数据属性：

- **名称：**跨度的标签或标识符，指示正在执行的操作类型。
- **开始时间戳：**跨度开始的确切时间，为跟踪和性能分析提供时间上下文。
- **持续时间：**跨度操作所花费的总时间，从开始到结束进行测量。此指标有助于分析效率并识别潜在的性能瓶颈。
- **属性：**
 - **输入和输出：**输入到跨度的数据（例如，用户目标）以及相应的输出（例如，工具调用结果或最终结果）。
 - **错误类型、消息和回溯：**关于遇到的任何错误的信息，包括错误类型、描述性消息以及用于调试的回溯详情。
 - **度量指标：**与跨度相关的定量数据，例如输入令牌、输出令牌、评估指标、监控指标，用于测量和跟踪成本使用情况以及智能体性能。

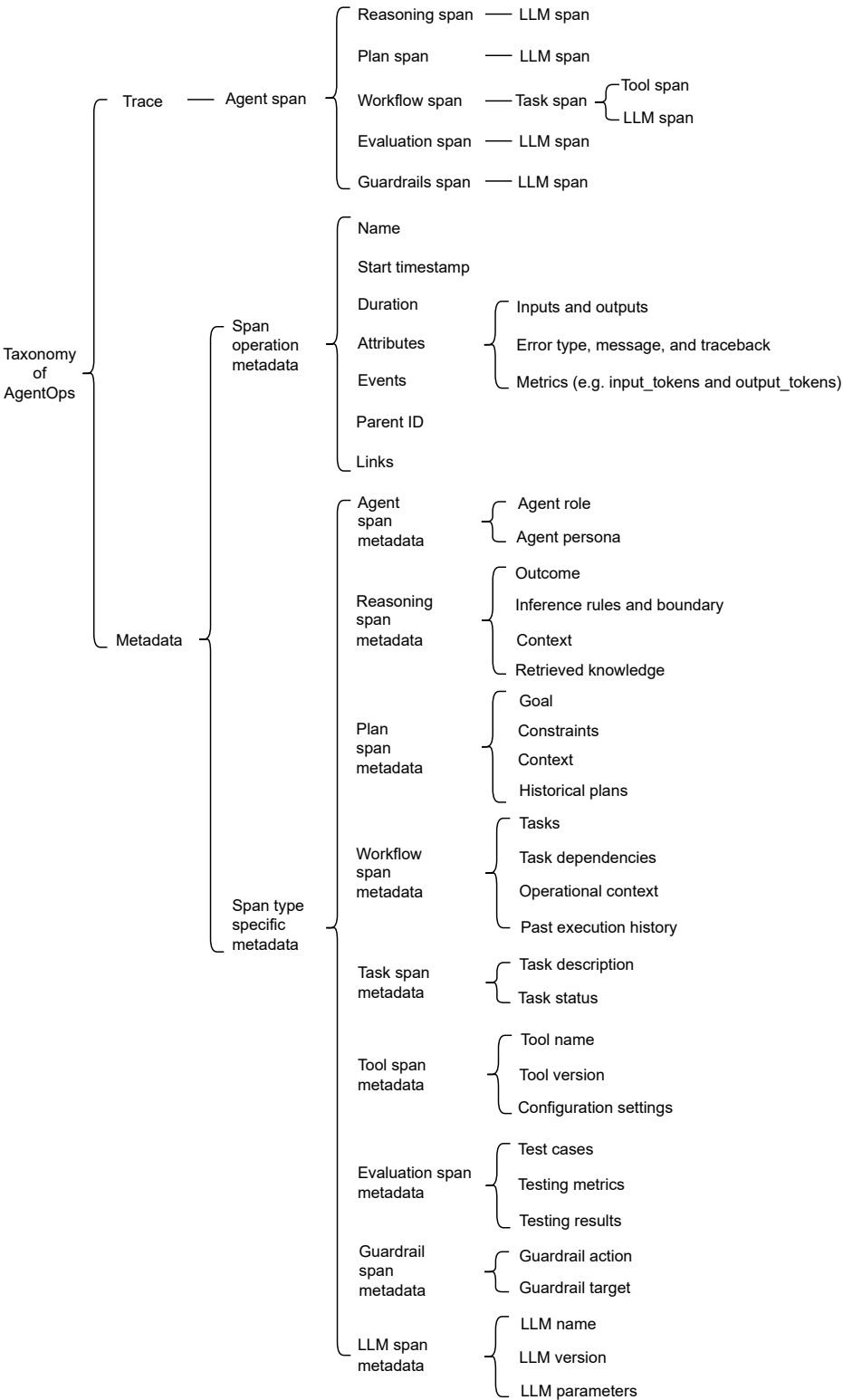


Figure 3: Taxonomy of AgentOps

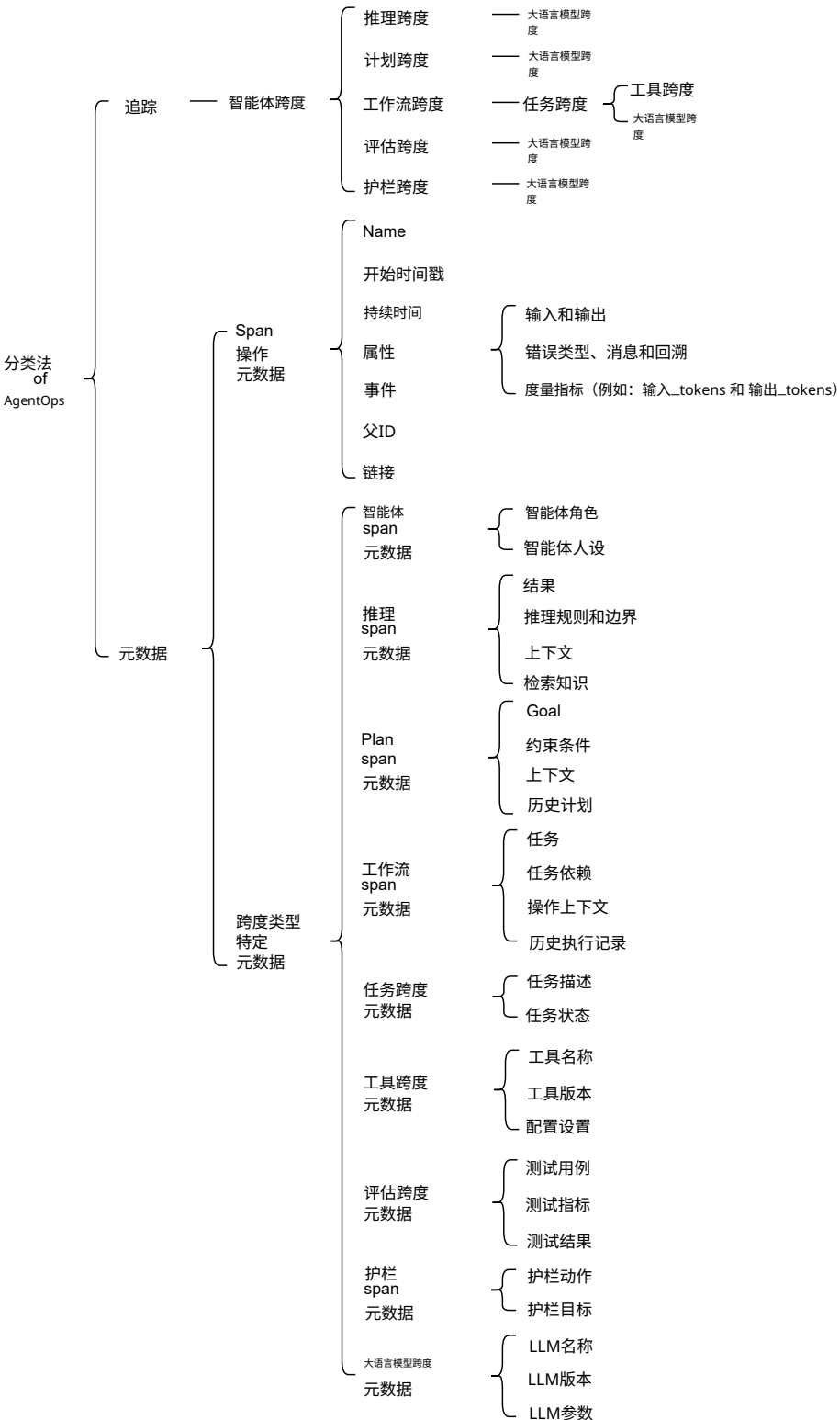


图 3: AgentOps分类法

- **Events:** Specific occurrences or a detailed timeline of actions and transitions within each span, providing detailed insights into the span’s activities or any notable events during its lifecycle.
- **Parent ID:** An identifier that links this span to its parent span, establishing a hierarchical relationship between spans and helping to track nested operations.
- **Links:** Connections to other spans or external references, which can help in understanding dependencies and relationships between different spans in a complex workflow.

In the context of agent spans, where a trace consists of a series of nested spans representing different operational spans or runs, each type of span involved in agent span plays a distinct role. The agent span includes the following nested spans:

- **Agent Span:** The agent span metadata includes the agent’s role and persona, which significantly influence how the agent performs tasks, interacts with users and makes decisions. The agent span meta data includes the following.
 - **Agent Role:** The scope or responsibility of the agent.
 - **Agent Persona:** The behavioral characteristics and interaction style the agent adopts.
- **Reasoning Span:** Reasoning span, captures the reasoning processes of the agent. Reasoning Span metadata including:
 - **Context:** The relevant information or situational data that informs the reasoning process. This may include inputs from previous spans or external factors that influence the agent’s reasoning.
 - **Retrieved Knowledge:** Information or data that the agent retrieves and references during reasoning, possibly from external sources or memory, to support its reasoning.
 - **Inference Rules and Boundary:** The logical rules, constraints, or boundaries applied during the reasoning. This could include specific guidelines or conditions under which the agent performs reasoning.
 - **Outcome:** The thoughts generated or conclusion reached after the reasoning process.
- **Planning Span:** Planning span records the planning phase, where the agent outlines the steps or objectives required to achieve the goal. It defines the intended sequence of operations, setting up a structured path for the agent’s activities. Planning Span metadata including:
 - **Goal:** The specific objective or desired outcome that the agent intends to achieve through this plan.
 - **Constraints:** The limitations or restrictions within which the plan must operate. These constraints can include time limits, resource limitations, or predefined rules that shape the planning process.
 - **Context:** Relevant situational or environmental information that informs the plan.
 - **Historical Plans:** Records of previous plans that may influence the current planning process. This includes past strategies or actions taken in similar contexts, which can provide valuable insights or best practices for the current plan.
- **Workflow Span:** Within the workflow span, the trace documents the organization of tasks through task spans and tool Spans. The workflow span, supported by an LLMspan, details how individual tasks are broken down and managed. The task span represents specific actions within the workflow, while the tool span (nested within the task span) logs interactions with tools or external resources the agent uses to fulfill specific parts of the workflow. The metadata of the workflow span includes the following.
 - **Tasks:** A list of task span that need to be completed as part of the workflow. This defines the specific actions or steps the agent needs to perform within this span.
 - **Task Dependencies:** Information about dependencies between tasks, indicating the order in which tasks should be executed or any prerequisites required for specific tasks. This helps manage the sequence and ensures that tasks are executed in a logical, efficient manner.
 - **Operational Context:** The situational information or environment details relevant to executing the workflow. This can include real-time conditions, status updates from other spans, or external factors that might influence task execution.
 - **Past Execution History:** Records of previous executions of similar workflows or tasks in (long-term) memory module, which can provide insights into best practices, potential pitfalls, or optimisation opportunities for the current workflow.
- **Task Span:** The task span represents a discrete unit of work or action within a workflow. It is a fundamental part of the workflow structure, defining individual tasks that the agent needs to execute in a sequence or parallel arrangement. The task span metadata includes:

- 事件：每个跨度内特定事件或操作与转换的详细时间线，为该跨度的活动或• 其生命周期内任何值得注意的事件提供详细见解。
- 父ID：一个标识符，将该跨度• 链接到其父跨度，从而在跨度之间建立层级关系，并有助于跟踪嵌套操作。

- 链接：与其他跨度或外部引用的连接，有助于理解复杂工作流程中不同跨度之间的依赖关系和关联。

在智能体跨度的上下文中，当追踪{v1}由一系列代表不同操作跨度或运行的嵌套跨度组成时，参与智能体跨度的每种跨度类型都扮演着独特的角色。智能体跨度包括以下嵌套跨度：

- 智能体跨度：智能体跨度元数据包括智能体的角色和人设，这显著影响智能体如何执行任务、与用户互动以及做出决策。智能体跨度元数据包括以下内容。
 - 智能体角色：智能体的范围或职责。
 - 智能体人设：智能体所采用的行为特征和交互风格。
- 推理跨度：推理跨度，捕捉智能体的推理过程。推理跨度的元数据包括：
 - 上下文：影响推理过程的相关信息或情境数据。这可能包括来自之前跨度的输入或影响智能体推理的外部因素。
 - 检索知识：智能体在推理过程中检索和参考的信息或数据，可能来自外部来源或记忆，以支持其推理。
 - 推理规则和边界：在推理期间应用的逻辑规则、约束条件或边界。这可能包括智能体执行推理时的特定指南或条件。
 - 结果：推理过程后生成的思想或得出的结论。
- 规划跨度：规划跨度记录规划阶段，在该阶段中，智能体概述实现目标所需的步骤或目标。它定义了操作的预期序列，为智能体的活动建立结构化的路径。规划跨度的元数据包括：
 - 目标：智能体希望通过此计划实现的特定目标或期望结果。
 - 约束条件：计划必须在其中运行的局限性或限制。这些约束条件可以包括时间限制、资源限制或塑造规划过程的预定义规则。
 - 上下文：为计划提供信息的相关情境或环境信息。
 - 历史计划：以往计划的记录，可能影响当前的规划过程。这包括在类似上下文中采取的过往策略或行动，可为当前计划提供有价值的见解或最佳实践。
- 工作流跨度：在工作流跨度内，追踪文档记录了通过任务跨度和工具跨度对任务的组织。由大语言模型跨度支持的工作流跨度，详细说明各个任务如何分解和管理。任务跨度代表 workflow 中的特定操作，而工具跨度（嵌套在任务跨度内）记录智能体使用工具或外部资源以完成 workflow 特定部分的交互。工作流跨度的元数据包括以下内容。
 - 任务：需要在作为工作流一部分完成的任务跨度列表。这定义了智能体需要在此跨度内执行的具体操作或步骤。
 - 任务依赖：关于任务间依赖性的信息，指示任务应执行顺序或特定任务所需的先决条件。这有助于管理顺序，并确保任务以逻辑且高效的方式执行。
 - 操作上下文：与执行 workflow 相关的情境信息或环境细节。这可能包括实时状态、来自其他跨度的状态更新，或可能影响任务执行的外部因素。
 - 历史执行记录：在（长期）记忆模块中类似 workflow 或任务过往执行的记录，可为当前 workflow 提供最佳实践、潜在陷阱或优化机会的见解。
- 任务跨度：任务跨度表示 workflow 中的一个离散工作单元或动作。它是 workflow 结构的基本组成部分，定义了智能体需要以顺序或并行方式执行的各项任务。任务跨度元数据包括：

- **Task Description:** Specific information about the task to be performed, including task objectives, instructions, and parameters needed for execution.
- **Task Status:** The current status (e.g., pending, in progress, completed) and the result of the task, which could include success, failure, or a specific output generated by the task.
- **Tool Span:** The tool span represents interactions with external tools or resources that assist in task execution. This span captures details about tools utilized by the agent, logging their configurations, responses, and any intermediate outputs. Tool Span metadata includes:
 - **Tool Name:** The name of the tool.
 - **Tool Version:** The version of the tool.
 - **Configuration Settings:** Parameters or settings configured for the tool during its use, such as version restriction, input formats, timeouts, or resource limits that may influence its behavior.
- **Evaluation Span:** The evaluation span assesses the correctness and quality of the agent’s actions and outputs performance against predefined criteria. It verifies whether the agent’s actions align with expectations or quality standards, providing a feedback mechanism to ensure that outputs meet the intended goals. The metadata of the evaluation span includes the following.
 - **Test Cases:** Specific scenarios or conditions under which the agent’s performance or outputs are evaluated. These cases provide a structured way to assess the agent’s actions and outcomes against expected behavior [8].
 - **Testing Metrics:** Quantitative or qualitative measures used to assess the agent’s performance. These metrics could include accuracy, efficiency, relevance, or other criteria that define the quality of the agent’s output.
 - **Testing Results:** The actual outcomes or findings from the evaluation process, indicating how well the agent’s performance aligns with the expected standards defined in the test cases and metrics.
- **Guardrail Span:** The guardrail span defines the guardrails applied to ensure the agent’s operations are aligned with expected governance requirements [9]. It helps prevent errors or unintended actions by setting boundaries on the agent’s behavior. Guardrail span metadata includes:
 - **Guardrail Actions:** Guardrails actions triggered, such as block, validation, filter, etc.
 - **Guardrail Targets:** Specific agent artifacts that guardrails are applied, such as goals and tools.
- **LLM Span:** The LLM Span captures interactions with an LLM, where the model processes language-based inputs to generate responses or insights. This span is essential for tasks involving natural language understanding, generation, or interpretation. LLM Span metadata could includes:
 - **LLM Name:** The name of the LLM.
 - **LLM Version:** The version of the LLM.
 - **LLM Parameters:** Settings applied to the LLM, such as temperature (affecting randomness), max_tokens (limiting response length), and other relevant hyper-parameters that shape the output.

5 Threats to Validity

Tool Selection Limitations: Due to the rapid proliferation of various tools and AI platforms, it is possible that not all relevant AgentOps tools were identified. To address this limitation, we selected tools from multiple data sources. The identified tools include both open-source AgentOps tools, such as AgentOps and Langfuse, as well as commercial observability platforms like Datadog.

Data Coverage Limitations: The comprehensive overview of traceable artifacts throughout the AgentOps life-cycle provided in this work may not encompass all possible data attributes related to AI agents. To ensure broader coverage of important traceable data across the entire life-cycle of an agent and enrich on the data attributes outlined in our paper, we have drawn on some relevant academic literature [9–11] to support our findings as well. However, some potentially valuable data, such as trace links and interactions between different steps, may have been missed. Future work will aim to explore these gaps further.

6 Conclusion

In this paper, we proposed AgentOps, a specialised DevOps paradigm tailored for LLM agents to enable observability. Through a systematic mapping study of existing AgentOps relevant tools, we first proposed an entity-relationship model

– 任务描述: 关于待执行任务的具体信息, 包括任务目标、指令以及执行所需的参数。– 任务状态: 任务的当前状态 (例如: 待处理、进行中、已完成) 及任务结果, 可能包括成功、失败或任务生成的特定输出。

• 工具跨度: 工具跨度表示用于辅助任务执行的外部工具或资源的交互。该跨度捕获了智能体所使用的工具的详细信息, 记录其配置、响应以及任何中间输出。工具跨度元数据包括:

– 工具名称: 工具的名称。– 工具版本: 工具的版本。– 配置设置: 在使用该工具期间为其配置的参数或设置, 例如版本限制、输入格式、超时或可能影响其行为的其他资源限制。

• 评估跨度: 评估跨度根据预定义标准评估智能体行为和输出的正确性与质量。它验证智能体的行为是否符合预期或质量标准, 并提供一种反馈机制, 以确保输出满足既定目标。 The

metadata of the evaluation span includes the following.

– 测试用例: 用于评估智能体性能或输出的特定场景或条件。这些用例提供了评估智能体操作和结果是否符合预期行为 [8] 的标准化方式。– 测试指标: 用于评估智能体性能的定量或定性措施。这些指标可以包括准确性、效率、相关性或其他定义智能体输出质量的准则。– 测试结果: 评估过程中的实际结果或发现, 表明智能体的性能与测试用例和指标中定义的预期标准的符合程度。

• 护栏跨度: 护栏跨度定义了为确保智能体操作符合预期治理要求 [9] 而应用的护栏。它通过设定智能体行为的边界来帮助防止错误或意外操作。护栏跨度元数据包括:

– 护栏操作: 触发的护栏操作, 例如阻止、验证、过滤等。– 护栏目标: 应用了护栏的特定智能体工件, 例如目标和工具。• 大语言模型跨度: 大语言模型跨度捕获与大语言模型的交互, 其中模型处理基于语言的输入以生成响应或见解。此跨度对于涉及自然语言理解、生成或解释的任务至关重要。大语言模型跨度的元数据可包括:

– LLM名称: 大语言模型的名称。– LLM版本: 大语言模型的版本。– LLM参数: 应用于大语言模型的设置, 例如温度 (影响随机性)、最大词元 (限制响应长度) 以及其他影响输出的相关超参数。

5 对有效性的威胁

工具选择局限性: 由于各种工具和人工智能智能体平台的迅速普及, 可能未识别出所有相关的AgentOps工具。为解决这一局限性, 我们从多个数据源中选择了工具。识别出的工具包括开源AgentOps工具 (如AgentOps和Langfuse), 以及Datadog等商业可观测性平台。

数据覆盖局限性: 本工作中提供的智能体整个生命周期内可追踪工件的综合概述, 可能并未涵盖与人工智能智能体相关的所有可能的数据属性。为确保更广泛地覆盖智能体整个生命周期中重要的可追踪数据, 并丰富我们论文中概述的数据属性, 我们还参考了一些相关的学术文献 [9–11] 以支持我们的发现。然而, 一些可能有价值的数据, 例如不同步骤之间的追踪链接和交互, 可能已被遗漏。未来工作将旨在进一步探索这些差距。

6 结论

在本文中, 我们提出了AgentOps, 这是一种专为大语言模型智能体量身定制的DevOps范式, 旨在实现可观测性。通过对现有AgentOps相关工具进行系统映射研究, 我们首先提出了一种实体关系模型

to understand the key agent artifacts and their relationship. Then we presented a comprehensive taxonomy of AgentOps, offering a structured template for developers to design and implement AgentOps tools. Future research will focus on validating the proposed taxonomy through real-world case studies and the development of a AgentOps tool prototype.

References

[1] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill *et al.*, “On the opportunities and risks of foundation models,” *arXiv preprint arXiv:2108.07258*, 2021.

[2] Y. Liu, S. K. Lo, Q. Lu, L. Zhu, D. Zhao, X. Xu, S. Harrer, and J. Whittle, “Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.10467>

[3] Q. Lu, L. Zhu, X. Xu, Z. Xing, S. Harrer, and J. Whittle, “Towards responsible generative ai: A reference architecture for designing foundation model based agents,” in *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2024, pp. 119–126.

[4] Q. Lu, L. Zhu, J. Whittle, X. Xu *et al.*, *Responsible AI: Best Practices for Creating Trustworthy AI Systems*. Addison-Wesley, 2023.

[5] L. Bass, Q. Lu, I. Weber, and L. Zhu, *Engineering AI Systems: Architecture and DevOps Essentials*. Addison-Wesley, 2025.

[6] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering version 2.3,” Software Engineering Group, School of Computer Science and Mathematics, Keele University and Department of Computer Science University of Durham, Tech. Rep., 2007.

[7] V. Garousi, M. Felderer, and M. V. Mäntylä, “Guidelines for including grey literature and conducting multivocal literature reviews in software engineering,” *Inf. Softw. Technol.*, vol. 106, pp. 101–121, 2019. [Online]. Available: <https://doi.org/10.1016/j.infsof.2018.09.006>

[8] B. Xia, Q. Lu, L. Zhu, Z. Xing, D. Zhao, and H. Zhang, “An evaluation-driven approach to designing llm agents: Process and architecture,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.13768>

[9] M. Shamsujjoha, Q. Lu, D. Zhao, and L. Zhu, “Designing multi-layered runtime guardrails for foundation model based agents: Swiss cheese model for ai safety by design,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.02205>

[10] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, P. S. Dulepet, S. Vidyadhara, D. Ki, S. Agrawal, C. Pham, G. Kroiz, F. Li, H. Tao, A. Srivastava, H. D. Costa, S. Gupta, M. L. Rogers, I. Goncarencu, G. Sarli, I. Galynker, D. Peskoff, M. Carpuat, J. White, S. Anadkat, A. Hoyle, and P. Resnik, “The prompt report: A systematic survey of prompting techniques,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.06608>

[11] A. Chan, C. Ezell, M. Kaufmann, K. Wei, L. Hammond, H. Bradley, E. Bluemke, N. Rajkumar, D. Krueger, N. Kolt *et al.*, “Visibility into ai agents,” in *The 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2024, pp. 958–973.

以理解关键智能体工件及其关系。随后，我们呈现了AgentOps分类法，为开发者设计和实现AgentOps工具提供了一个结构化模板。未来的研究将专注于通过真实世界案例研究验证所提出的分类法，以及开发AgentOps工具原型。

References

[1] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill *et al.*, “On the opportunities and risks of foundation models,” arXiv预印本 arXiv:2108.07258, 2021.[2]Y. Liu, S. K. Lo, Q. Lu, L. Zhu, D. Zhao, X. Xu, S. Harrer, and J. Whittle, “智能体设计模式目录：基于基础模型的智能体的架构模式合集,” 2024年. [在线]. 可访问: <https://arxiv.org/abs/2405.10467>[3] Q. Lu, L. Zhu, X. Xu, Z. Xing, S. Harrer, and J. Whittle, “迈向负责任的生成式人工智能：为设计基于基础模型的智能体构建参考架构”，收录于IEEE第21届软件架构国际会议伴侣会（ICSA-C）。IEEE, 2024年, 第119–126页.[4] Q. Lu, L. Zhu, J. Whittle, X. Xu *et al.*, 负责任的AI：创建可信AI系统的最佳实践。Addison-Wesley出版社, 2023.[5] L. Bass, Q. Lu, I. Weber, and L. Zhu, 人工智能系统工程：架构与DevOps essentials。Addison-Wesley出版社, 2025.[6] B. Kitchenham and S. Charters, “软件工程中进行系统性文献综述的指南 版本2.3”，软件工程研究组，基尔大学计算机与数学学院与杜伦大学计算机系，技术报告，2007.[7] V. Garousi, M. Felderer, and M. V. Mäntylä, “在软件工程中包含灰色文献并进行多声部文献综述的指南”，《信息技术与应用软件》，卷106, 第101–121页, 2019. [在线]. 可访问: <https://doi.org/10.1016/j.infsof.2018.09.006>[8] B. Xia, Q. Lu, L. Zhu, Z. Xing, D. Zhao, and H. Zhang, “一种评估驱动的大语言模型智能体设计方法：流程与架构”，2024年. [在线]. 可访问: <https://arxiv.org/abs/2411.13768>[9]M. Shamsujjoha, Q. Lu, D. Zhao, and L. Zhu, “为基于基础模型的智能体设计多层运行时护栏：用于设计安全的人工智能的瑞士奶酪模型”，2024年. [在线]. 可访问: <https://arxiv.org/abs/2408.02205>[10] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, P. S. Dulepet, S. Vidyadhara, D. Ki, S. Agrawal, C. Pham, G. Kroiz, F. Li, H. Tao, A. Srivastava, H. D. Costa, S. Gupta, M. L. Rogers, I. Goncarencu, G. Sarli, I. Galynker, D. Peskoff, M. Carpuat, J. White, S. Anadkat, A. Hoyle, and P. Resnik, “《提示词报告》：提示技术的系统性综述”，2024年. [在线]. 可访问: <https://arxiv.org/abs/2406.06608>[11] A. Chan, C. Ezell, M. Kaufmann, K. Wei, L. Hammond, H. Bradley, E. Bluemke, N. Rajkumar, D. Krueger, N. Kolt *et al.*, “人工智能智能体的可见性”，收录于2024年ACM公平性、问责制与透明度会议，2024年, 第958–973页。